گزارش تمرین اول
سرور تراکنش بانکی کوچک

درس سیستم‌های توزیع‌شده • کوروش اعتضاد (۴۰۴۷۲۳۰۱۸) • ۲۹/۰۷/۱۴۰۵

۱. مقدمه و نگاه کلی

این پروژه یک سیستم بانکی کوچک و **شبیه‌سازی‌شده** به شکل client–server است؛ تمام حساب‌ها و موجودی‌ها ساختگی‌اند و هیچ اطلاعات بانکی واقعی در کد یا داده‌های آزمایشی وجود ندارد. هدف، کار عملی با مفاهیم درس است: ارتباط client و server، رفتار server زیر بار چند client هم‌زمان، نگه‌داری shared state، و واکنش درست به requestهای نامعتبر. مدیریت خطا و logging نیز بخشی از نمره‌ی این تمرین است.

سیستم دو بخش اصلی دارد:

- **server** (server/bank_server.py و server/bank.py): حساب‌های شبیه‌سازی‌شده را در یک SQLite database نگه می‌دارد و با response HTTP protocol می‌دهد.
- **client** (client/bank_client.py): یک برنامه‌ی خط فرمان که می‌تواند یک command را جداگانه بفرستد یا در حالت پوسته‌ی تعاملی کار کند.

کد تنها از کتابخانه‌ی استاندارد پایتون ۹.۳ به بالا استفاده می‌کند (http.server، sqlite3، json، threading، argparse) و به هیچ بسته‌ی جانبی نیاز ندارد.

۲. روش ارتباط و قالب پیام

برای ارتباط، **HTTP/1.1** انتخاب شده است: یک protocol جاافتاده با واحد طبیعی «response/request» که logging هر request و response را تمیز و مرتب می‌کند. نقطه‌ی ورود اصلی / POST با بدنه‌ی JSON است و GET /health یک بررسی ساده‌ی زنده‌بودن server است.

request	{"action":"withdraw","account":"A1001","amount":200}
response موفق	{"status":"success","account":"A1001","balance":800}
response خطا	{"status":"error","error":"insufficient_funds",…}

هر response یک X-Request-Id header دارد که با id همان request در فایل log یکسان است و ردگیری یک request خاص را ساده می‌کند. **تمام مبالغ در داخل سیستم به‌صورت عدد صحیح «سنت» ذخیره می‌شوند** تا خطای گرد کردن اعداد اعشاری (استاندارد IEEE-754) موجودی را خراب نکند. کدهای وضعیت یک قرارداد ساده دارند: 400 وقتی بدنه اصلاً قابل‌تجزیه نیست، 422 وقتی JSON معتبر است ولی request رد می‌شود، و 503 وقتی سقف clientهای هم‌زمان پر است.

۳. مدیریت clientهای هم‌زمان و shared state

server روی ThreadingHTTPServer ساخته شده است؛ یعنی برای هر اتصال یک **thread جدا** اجرا می‌شود و چند client در همان

لحظه سرو می‌شوند. یک شیء مشترک Bank نگه‌دارنده‌ی حساب‌هاست و تمام threadها روی همان یک شیء کار می‌کنند.

حداکثر تعداد clientهای هم‌زمان قابل تنظیم است و هرگز در کد سخت نشده. مقدار به ترتیب اولویت فزاینده از سه منبع خوانده می‌شود: فایل server/config.json، متغیرهای محیطی (مثل BANK_MAX_CLIENTS)، و پارامترهای خط فرمان (--max-clients). این محدودیت با یک BoundedSemaphore و در لحظه‌ی پذیرش اتصال اعمال می‌شود (نه داخل مدیریت‌کننده)، تا شمار «clientهای وصل‌شده» محدود شود نه «requestهای در جریان». اتصال اضافی بلافاصله server_busy 503 می‌گیرد و بسته می‌شود.

۴. دسترسی هم‌زمان صحیح به یک حساب

تمام state حساب‌ها در SQLite است و هر عملیات تغییردهنده داخل یک transaction واحد BEGIN IMMEDIATE … COMMIT اجرا می‌شود؛ یک write lock سراسری (RLock) نیز transactionهای نوشتن روی اتصال مشترک را serialize می‌کند. نکته‌ی کلیدی BEGIN IMMEDIATE است: lock نوشتن را **از همان ابتدا** می‌گیرد، نه در لحظه‌ی نوشتن نهایی. بنابراین توالی «خواندن-تغییر-نوشتن» (خواندن موجودی، بررسی کفایت، کسر مبلغ) با توالی مشابه نویسنده‌ی دیگر در هم نمی‌آمیزد. نتیجه این‌که عملیات هم‌زمان روی یک حساب **مثل اجرای یکی‌یکی در یک ترتیب معین** عمل می‌کنند (serializability)؛ پس نه **به‌روزرسانی گمشده** رخ می‌دهد و نه موجودی هرگز **منفی** می‌شود.

اسکریپت tests/test_concurrency.py تعداد ۶۰ برداشت هم‌زمان ۰۰.۱۰ روی حسابی با موجودی ۰۰.۱۰۰۰ می‌فرستد و بررسی می‌کند موجودی نهایی دقیقاً ۰۰.۴۰۰ باشد:

threads=60 succeeded=60 insufficient=0 | final balance = 400.0 | PASS

۵. ویژگی‌های ACID

هر عملیات تغییردهنده‌ی موجودی یک transaction است؛ جدول زیر نشان می‌دهد هر ویژگی چگونه تضمین می‌شود.

ویژگی	نحوه‌ی تضمین
Atomicity	هر عملیات یک transaction واحد SQLite است؛ هر شکست (ورودی نامعتبر، موجودی ناکافی، استثنا) باعث ROLLBACK می‌شود، پس عملیات نیمه‌کاره اثری ندارد. در انتقال وجه، کسر و افزودن هر دو در یک transactionاند، پس با هم انجام می‌شوند یا با هم لغو.
Consistency	مبلغ پیش از هر نوشتن اعتبارسنجی می‌شود و موجودی در همان transaction بررسی و به‌روز می‌شود؛ قید CHECK (balance >= 0) آخرین سد دفاعی است. کل پول سیستم فقط از راه واریز و برداشت تغییر می‌کند.
Isolation	ترکیب BEGIN IMMEDIATE و write lock، توالی‌های خواندن-تغییر-نوشتن را نسبت به هم atomic می‌کند؛ هیچ threadی کار ناتمام دیگری را نمی‌بیند و نتیجه معادل یک ترتیب serial است.
Durability	database در حالت **WAL** با synchronous=FULL اجرا می‌شود، پس SQLite در COMMIT عمل fsync را انجام می‌دهد. server **تنها پس از commit موفق** response موفقیت می‌فرستد؛ بنابراین transaction تأییدشده از سقوط جان سالم به در می‌برد.

۶. مدیریت خطا

requestهای نامعتبر پیام خطای معنادار با کد پایدار می‌گیرند و server به کارش ادامه می‌دهد؛ استثنای غیرمنتظره به internal_error تبدیل می‌شود و هرگز فرایند را از کار نمی‌اندازد.

اسکریپت tests/test_errors.py همه‌ی موارد بالا را می‌آزماید و در پایان تأیید می‌کند که server همچنان responseدهنده است (۱۱ مورد،

مورد	نمونه	کد خطا
حساب ناموجود	Z9999	unknown_account
موجودی ناکافی	برداشت ۵۰۰۰ از ۸۰۰	insufficient_funds
مبلغ منفی / صفر / غیرعددی	-50، 0، "abc"	invalid_amount
عمل پشتیبانی‌نشده	"frobnicate"	unsupported_action
فیلد گمشده	برداشت بدون amount	missing_field
request بدشکل	JSON خراب یا بایت تصادفی	malformed_request

همه PASS).

۷. Logging

هر request و responseش به‌صورت یک **خط JSON** در server/requests.log نوشته و روی کنسول هم چاپ می‌شود. هر رکورد شامل همه‌ی فیلدهای خواسته‌شده است: request id، client IP، عمل، account id، زمان، نتیجه و latency؛ به‌علاوه‌ی کد HTTP و موجودی نتیجه. requestهای GET /health و مسیر ناشناس (۴۰۴) هم به همین شکل ثبت می‌شوند، پس X-Request-Id header همیشه معادل یک خط log دارد. رویدادهای server_start و server_stop (همراه تصویر حساب‌ها) هم ثبت می‌شوند.

{"ts":"2026-10-12T10:41:07.512Z","req":"7f3a","ip":"192.168.64.129",
"action":"withdraw","account":"A1001","result":"success",
"http":200,"latency_ms":3.0,"balance":800.0}

۸. استقرار روی دو ماشین مجازی

client و server روی **دو ماشین مجازی جدا** زیر VMware و روی یک شبکه‌ی **Host-only** یا **Bridged** اجرا می‌شوند. client به **IP واقعی** server وصل می‌شود، نه localhost.

```
# server VM (192.168.64.130):
python3 server/bank_server.py --host 0.0.0.0 --port 8080 --max-clients 50 --db server/bank.db
# client VM (192.168.64.129):
python3 client/bank_client.py --host 192.168.64.130 balance A1001
```

گزینه‌ی 0.0.0.0 host-- باعث می‌شود server روی همه‌ی رابط‌های شبکه گوش دهد تا از ماشین دیگر قابل‌دسترسی باشد. اگر فایروال میزبان فعال باشد، درگاه با sudo ufw allow 8080/tcp باز می‌شود.

۹. بخش امتیازی: خوشه‌ی backup/primary مقاوم به خرابی

در این بخش، سیستم به یک خوشه با دو نمونه‌ی server (primary و backup) گسترش یافته تا با از کار افتادن server اول، سرویس روی server دوم ادامه یابد و **هیچ transactionی گم یا دوبار اعمال نشود**. هر دو node یک برنامه‌ی واحد (server/cluster_server.py) را با نقش متفاوت اجرا می‌کنند.

protocol ثبت یک نوشتن: command primary را اعتبارسنجی و داخل یک transaction باز اعمال می‌کند؛ سپس رکورد log را به

backup می‌فرستد و **منتظر تأیید** می‌ماند؛ و تنها پس از تأیید، محلی COMMIT می‌کند و به response client می‌دهد. اگر تکرار شکست بخورد، transaction برگردانده می‌شود؛ پس نوشتنی که فقط روی یک ماشین است هرگز تأیید نمی‌شود.

exactly-once: هر request نوشتن یک request_id پایدار دارد و جدول applied_requests نگاشت id به response ذخیره‌شده را نگه می‌دارد. clientی که response نگرفته، همان id را به primary جدید می‌فرستد و **همان response اصلی** را می‌گیرد؛ پس عمل دوباره اعمال نمی‌شود.

انتقال کنترل: هر node هر ثانیه حال همتا را می‌پرسد. با از کار افتادن A، backup پس از یک مهلت کوتاه (پیش‌فرض ۳ ثانیه) با شماره‌ی term بالاتر خودکار primary می‌شود. client با هر دو آدرس اجرا می‌شود (--hosts A:8080,B:8081) و با همان request_id روی node responseدهنده تلاش می‌کند.

خواندن در برابر نوشتن: فقط leader می‌تواند موجودی را تغییر دهد؛ نوشتن روی backup با not_primary 503 رد می‌شود. خواندن (balance) از هر nodeای سرو می‌شود، که بی‌خطر است چون تکرار هم‌زمان است و هر دو node داده‌ی یکسان دارند؛ تنها هنگام همگام‌سازی مجدد، backup ممکن است لحظه‌ای عقب باشد.

محدوده‌ی این بخش: این طراحی فقط «از کار افتادن» node را پوشش می‌دهد، نه «قطع شدن شبکه» را؛ چون با دو node هیچ طرف سومی برای داوری وجود ندارد. آن حالت به‌طور جداگانه در بخش Bonus Extreme حل شده است (بخش ۱۰).

آزمایش‌های این بخش: test_failover.py، test_exactly_once.py، test_client_failover.py و test_failover_under_load.py (کشتن server اول در حین نوشتن ۶ client هم‌زمان؛ همه‌ی ۳۰ request response داده می‌شوند و موجودی نهایی دقیق است).

۱۰. بخش امتیازی Bonus: Extreme — Network Partition و Split Brain

بخش پیشرفته یک حالت بنیادی‌تر را حل می‌کند: **قطع شدن شبکه بین دو server، در حالی که هر دو پردازه سالم و زنده‌اند.** این با «از کار افتادن node» یکی نیست، ولی از دید یک node دقیقاً همان‌طور به‌نظر می‌رسد: در هر دو حالت همتا response نمی‌دهد. پس منطق «همتا جواب نداد، پس من leader می‌شوم» هر دو طرف را هم‌زمان leader می‌کند؛ نتیجه دو نسخه‌ی واگرا از موجودی‌هاست (**Split Brain**). این محدودیت با دو رأی‌دهنده حل‌شدنی نیست و به یک رأی سوم نیاز دارد.

داور (Witness)

یک سرویس سوم (extremeBonus/witness_server.py) روی یک **ماشین سوم مستقل** اضافه شده که **هیچ داده‌ی حسابی نگه نمی‌دارد** و تنها به یک پرسش response می‌دهد: «آیا X node می‌تواند term leader شماره‌ی T باشد؟» داور در هر term **فقط به یک node** اجازه می‌دهد. تغییر کلیدی در منطق node این است: همتای جواب‌نداده دیگر به معنای «من می‌توانم leader شوم» نیست، بلکه به معنای «اول از داور بپرسم» است. اگر داور اجازه ندهد، node نوشتن را با not_primary 503 رد می‌کند و **فقط خواندنی** می‌ماند.

وضعیت	رفتار node
همتا در دسترس، همتا leader است	follower می‌شود و در صورت عقب‌بودن همگام می‌شود
همتا در دسترس، کسی leader نیست	primary تعیین‌شده از داور اجازه می‌گیرد
همتا قطع، مهلت تمام‌شده	request term بعدی؛ **مجاز شد** → leader تنها، **رد شد** → ماندن در حالت خواندنی
داور در دسترس نیست	**هرگز** خود را leader نمی‌کند؛ فقط خواندنی می‌ماند

ردیف آخر یک **انتخاب آگاهانه‌ی CAP** است، نه فراموشی: اگر node نتواند به داور برسد، نمی‌داند آیا طرف جداشده است یا نه، پس avail-ability را فدا می‌کند تا consistency حفظ شود. چون در هر term فقط یک مالک وجود دارد و شماره‌ی term هرگز کاهش نمی‌یابد، حداکثر

یک node می‌تواند بنویسد و log دو طرف واگرا نمی‌شود. تکرار نوشتن نیز مانند بخش پیشرفته **همگام** است، پس نوشتن تأییدشده روی هر دو طرف وجود دارد.

آزمایش‌ها

سرویس داور روی ماشین سوم اجرا می‌شود و دو server با witness-- به آن وصل می‌شوند. سناریوی partition با iptables (قطع ترافیک بین دو server، بدون کشتن هیچ پردازه‌ای) شبیه‌سازی می‌شود. اسکریپت extremeBonus/test_partition.py کل مسیر را خودکار می‌آزماید (حالت عادی → قطع لینک → بررسی عدم split brain → برگرداندن لینک → همگام‌شدن). خروجی کلیدی آن:

during the split: nodes claiming leadership: 1 | OK: no split brain
write to A -> http 200 | write to B -> http 503 (not_primary)
after healing: A hash=deb89c2793a553df = B hash=deb89c2793a553df | OK: logs converged
PARTITION TEST: ALL CHECKS PASSED

در حالت قطعی، A server با leader: true, solo: true تنها نویسنده می‌ماند و B server با reason: read_only:term_owned_by_A رد می‌شود؛ یعنی **فقط یکی** از دو طرف نوشتن را می‌پذیرد. برای اثبات این‌که داور همین کار را می‌کند و نه شانس، همان server با سوییچ no-witness-- هم اجرا می‌شود: در آن حالت **هر دو** node :leader: true می‌شوند و split brain بلافاصله برمی‌گردد.

۱۱. جمع‌بندی و نحوه‌ی اجرا

python3 server/bank_server.py --host 0.0.0.0 --port 8080 --max-clients 50 --db server/bank.db
./run_tests.sh

دستور دوم کل مجموعه‌ی آزمون‌ها را اجرا می‌کند. پروژه این خواسته‌ها را انجام می‌دهد: سه عمل پایه و انتقال وجه اضافی، هم‌زمانی با سقف قابل تنظیم، shared state و serializability، چهار ویژگی ACID، هر شش حالت خطا، logging کامل، و استقرار روی دو VM. بخش امتیازی پیشرفته (خوشه‌ی backup/primary) نیز پیاده و آزمایش شده است. علاوه بر آن، بخش Bonus Extreme رفتار سیستم را در برابر **network partition واقعی** تضمین می‌کند: با یک داور روی ماشین سوم، در هر term فقط یک node اجازه‌ی نوشتن دارد، پس split brain رخ نمی‌دهد و پس از برگشتن شبکه log دو طرف همگام می‌شود.

github: https://github.com/kouroshetezad/mini-banking-server